

BDH sub-design - Foundations

Document Control

Document Status*	N DESIGN COMPLETED
Purpose*	To describe: - How the BDH will have accounts split to meet GIRP compliance requirements - The data layers and databases that the BDH will have - The SDLC environments that the BDH will support - The new specification and deployment process for Zones
Design Authority Status & Date*	ENDORSED 06 DEC 2024 VIA CONFLUENCE/JIRA
Authors*	Lachlan Nunn (Accenture) Mughees Ahmed (Accenture) Fayaz Syed (Accenture) Abhijit Ghosh (Accenture) Ravi Karuturi (Accenture)
Jira Reference*	☒ APU-57 - DLD - Capability 1: Review account structure CLOSED ☒ APU-63 - DLD - Capability 2: Review future state tenancies and changes required CLOSED
References	File ingestion S3 design - Digital, Data & Analytics - BNZ Confluence
Notes	Endorsement achieved via Confluence and Jira 2026-07-28 Updated HLD Data Architecture diagram to reflect designed FDPs
Consulted	Paul Dudding Tracey Davis Dandan Cai Ivan Savchenko Chenna Pala Brendon Watson Stewart Robertson Anne Irwin Daniel Dove Yi-Jing Chung

Key Revision History

Date	Description	Change Impetus	Author(s)	Endorsement & Approval	Role /on behalf of	Endorsement date

05 Dec 2024	Sub-design created for Wave 1 of the 2024 Data Transformation programme.	BDH Platform Uplift Wave 1	Lachlan Nunn (Accenture) Mughees Ahmed (Accenture) Fayaz Syed (Accenture) Abhijit Ghosh (Accenture) Ravi Karuturi (Accenture)	Paul Dudding Amy Di Benedetto	Platform Architect Program Stream Lead	06 Dec 2024
24-March-2025	Updated 'Database Naming Convention' section	BDH Platform Uplift Wave 1	Fayaz Syed (Accenture)	Paul Dudding Amy Di Benedetto	Platform Architect Program Stream Lead	Already endorsed, needed alignment in documentation
12-Sep-2025	Added following sections for clarity Other Databases Comprehensive View of all the Databases in BDHUplift Logging Database Schemas	BDH Platform Uplift	Fayaz Syed	Paul Dudding	Platform Architect	Already endorsed, needed visibility in documentation
10-Oct-2025	Added section on <ZONE>__TRANSFORM__DEV	BDH Platform Uplift	Fayaz Syed	Paul Dudding	Platform Architect	Added for visibility and clarity
18-Dec-2025	Comprehensive View of all the Databases in BDHUplift Logging Database Schemas	BDH Platform Uplift	Fayaz Syed	Paul Dudding	Platform Architect	Corrected FINOPS details under logging (removed as this is under Platform Audit) and added ENV details for clarity

Table of Contents

- [Document Control](#)
- [Key Revision History](#)
- [Table of Contents](#)
- [Key Principles](#)
- [Key Assumptions](#)
- [Design Decisions](#)
- [Scope](#)
- [Out of Scope](#)
- [High Level Design](#)
 - [Data Architecture](#)
 - [BDH Zones](#)
- [Detailed Design](#)
 - [Account Structure](#)
 - [Future State Zones](#)

Key Principles

Specific principles for Foundations.

ID	Principle
P-1	Solution must simplify architecture componentry where possible.
P-2	Code should be promoted between environments, but data should not be promoted between environments.
P-3	Emit Data from Source as a Data Product, providing consistency for all consumers & data planes (where possible)
P-4	Split data at source where multi subject area e.g. loans vs TDs (where possible)
P-5	Any FDP van feed any EDP or BDP, regardless of zone, Any EDP can feed any other EDP or BDP

Key Assumptions

ID	Assumptions
A-1	Snowflake as primary data warehouse
A-2	Focus on structured and semi-structured data
A-3	End user access applies on Present layer using RBAC and PII inputs. The other layers will have persona role-based access for technical roles e.g. Data Engineer, Support, Data Analyst, Data Scientists

Design Decisions

Jira ID	Options/discussion	Design decision(s)	Justification
 APU-150 - Split BDH from a single account into two Snowflake accounts (PreProd and Prod) CLOSED	Options discussed included: a. Single account with logical separation (remain with current state with optimized controls) b. Split into two accounts for Non Prod and Prod	Decision made to split into two Snowflake accounts: #1: Dev, Test #2: UAT, Prod	Straightforward adherence to GIRP policies.
 APU-193 - Use of Four tier data layer design in BDH CLOSED	Option 1: Four-Layer Architecture (LANDING, RAW, FOUNDATION, CURATED) The initial design proposes a simplified architecture where the FOUNDATION & CURATED layer handles both foundation and enterprise data products and data exposure. This approach offers streamlined data flow, reduced complexity, and faster implementation. Option 2: Five-Layer Architecture (LANDING, RAW, FOUNDATION, CURATED, PRESENT) This design separates the final layer into CURATED (for enterprise data products and business logic) and PRESENT (purely for data exposure). While this adds an additional layer, it provides clear separation of concerns, better governance, and dedicated performance optimization.	Option 1 (Four-Layer Architecture) is selected to implement the data platform.	- Clear separation between data product creation and exposure - Simplified governance and ownership model - Manage the access on Foundation and Curated layer
 APU-195 - Specifications required for target state Zone design CLOSED	Multiple configuration approaches for all components which needed to be aligned to the data layering design and data engineering experience needed per the repo structure in the DevOps design.	See Zone section	- Repo / dbt / Airflow / dbt: aligned to subject areas per operating model and developer experience - Snowflake structure: breaks up the environments into manageable pieces
 APU-196 - Change from use of the word "Tenancy" to "Zone" CLOSED	Given confusion with the word Tenancy to describe the technical grouping that holds/supports subject areas and data products, proposed change to use of the word "Zone".	Agreement to use "Zone".	Clear break from "Tenancy" (which has historically caused confusion) but is a relevant descriptor for this purpose.
 APU-203 - Target state BDH database design CLOSED	Target state zone database design, at a high-level, had these options: - Vertically aligned - Databases are aligned to Zones (containing many data products). And permutations per SDLC environment e.g. Customer__Dev - Horizontally aligned - Databases are aligned to individual data products. And permutations per SDLC environment e.g. Customer__Account__Dev	Agreement to align to Zone	Provides the same RBAC control capability either option; but means that zones will have a standard deployment fitout for databases.

Scope

This High-Level Sub-Design will cover:

- The platform data architecture (including data layers and SDLC environments)
- The account structure
- BDH zones (previously known as tenancies) and how they are structured

Out of Scope

High Level Design

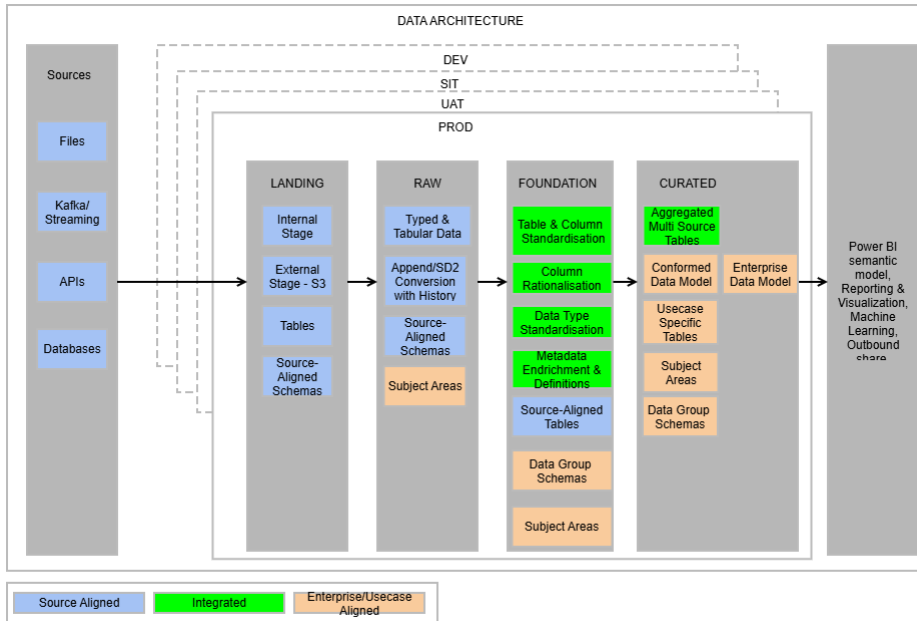
Data Architecture

In each environment, multiple Databases and Schemas are set up to facilitate the creation of end-to-end data pipelines. Moreover, we are implementing streamlined review and approval processes to enable the continuous deployment of new and changed Data Products.

The below diagram describes the Environments and Layers implemented, with further descriptions of each below.

At high level there are three types of data within an environment:

1. Source aligned - Data that uses terminology and is modelled as it appears in the source system
2. Integrated - Creates a unified and standardized representation of enterprise data; applying transformation rules for consistency.
3. Use case aligned - Data that uses terminology and is modelled to match the structure of the business, abstracting away from individual source system formats



Data Layers

Within each environment, data undergoes processing across the specified Data Layers below, transitioning from landing to refined data assets. Each layer serves as a reusable interface for subsequent transformations and utilization. These layers, deployed within individual environments, serve to establish the consistent pattern for the landing, keeping the source image, transforming, and presenting of data. They also facilitate data sharing and enable the efficient allocation of analytical resources.

Data Layer	Purpose
LANDING	Serves as an immutable storage layer to maintain exact copies of source data in its original format before any transformation.
RAW	Provides a structured historical storage layer implementing SCD Type 2 or append it according to source data pattern.
FOUNDATION	Cleansed and conformed data. Applies transformations to create conformed objects.
CURATED	Provides enterprise-wide, trusted data products through business rules and integrated data models.

Environments

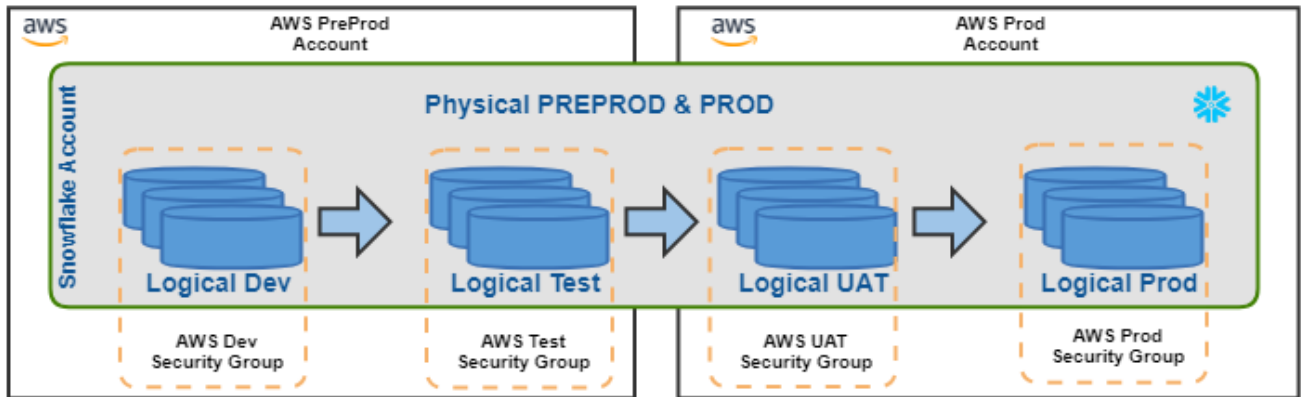
The four environments exist to support efficient development, testing, User acceptance and deployment of new and changed data objects. Development centres around Features, which are collections of new or changed functionality to be built and released as a single unit. While many Developers can contribute to a Feature, testing and review for a Feature occurs as a single process, allowing deployment as a single unit once the Feature is considered complete.

Data Environment	Details
Development	Developers create Features by adding and changing logic, and testing in developer aligned schemas within the environment.
Testing (SYST)	Features are tested in feature aligned testing schemas.
PPTE	Mirror production data, where business users can validate the data product features meet their requirements and aligned with their business processes e.g. performance testing etc. before promoting to production.
Production	The live environment where actual business activities occur, containing the actual data that supports the business processes, reporting and decision making.

Account structure

Initially BDH was launched with two Snowflake accounts - A Non-Prod (platforms) account for the BDH Platforms Team to build and deploy structure & features into a single Prod Snowflake account. The Prod account being for data product developers & consumers, containing the Software Development Lifecycle (SDLC) environments.

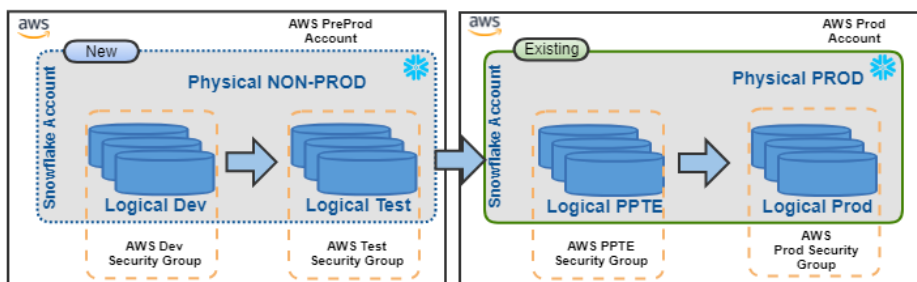
The new Snowflake account structure will retain the Non-Prod (platforms) account, add a new PreProd account and retain the current Prod account (3 account structure).



At a high level, the environments are logically separated from one another via:

- Separate Snowflake databases
- AWS security groups and common resources

As part of the BDH Platform Uplift, a decision was made to split the accounts in order to achieve the GIRP segregation requirement.



This will be achieved by:

1. Creation of a new "PreProd" Snowflake account
2. The establishment of Dev & Test Snowflake environments within PreProd
3. The Dev and Test AWS security groups (and any associated AWS configurations/resources such as S3) realigned to the new Dev and Test in PreProd
4. Original Dev & Test environments decommissioned in Snowflake

Note: migration of existing tenants is a known need but has not been scoped for Wave 1 delivery. For this reason, there will likely be a period of dual run where the original Dev & Test environments will need to be supported alongside of the new environments in PreProd.

BDH Zones

What is a Data Product?

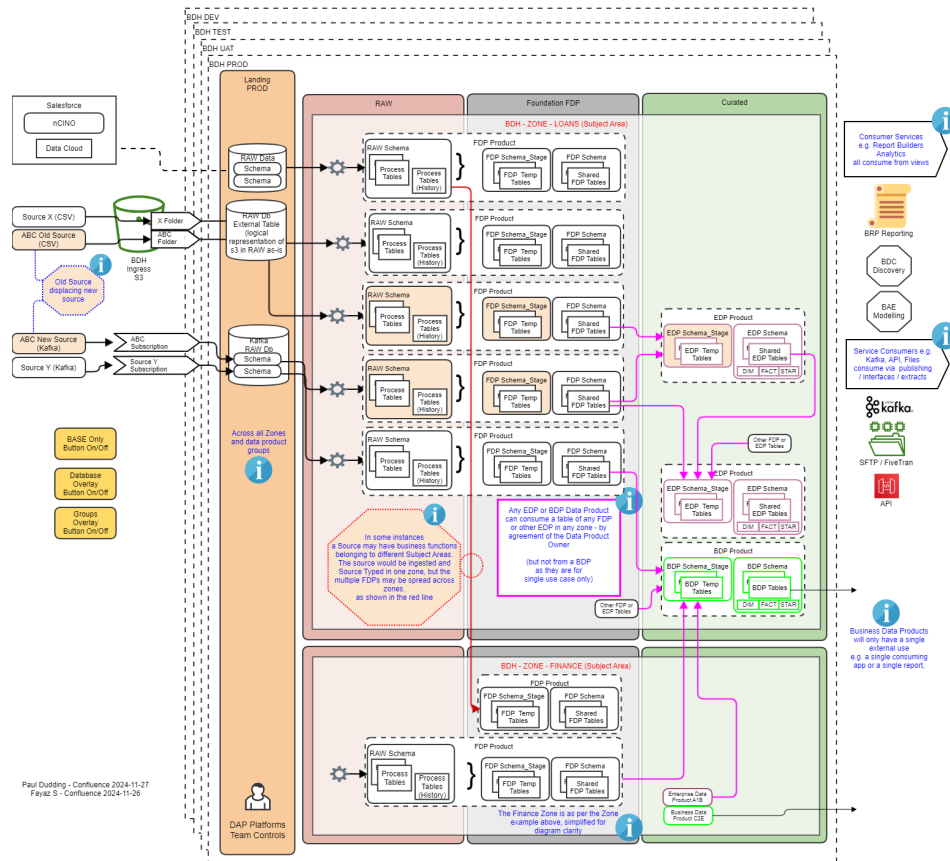
"A Data Product is a set of business data provided as a service for use in the data value chain."

- A Data Product is a component of the data architecture that contains a domain specific data set that is in a format that is easy to access, understand and query by a wide audience of analytics and reporting practitioners across BNZ.
- Designed to ease time-consuming 'data wrangling' and duplication of effort that data and analytics practitioners often undertake to prepare usable data sets..
- A Data Product should contain all the necessary transformation code, data, interface agreements & access policies it requires to function.
- Data publishing (also data publication) and Data servicing refers to the process of making data available to a wider audience. It involves preparing, organising, and releasing data in a structured and accessible format. This takes many forms, typically in relation to BDH it will be through Tables, Reports, and publishing through the multiple egress services.
- Further information related to this topic can be found in the [Architecture Vision - Data Hub](#).



Zone architecture

Zone architecture example



i An example with further detail (Loan to Value Ratio calculation) can be found here: [LVR Example 2](#)

Zone Specification

In order to separate and independently manage their groupings of data products (typically by subject area), each tenant will be provided with a standardized configuration that will be automatically deployed upon initiation.

Note: deployment of Zones is described in [BDH HL sub-design - DevOps - Digital, Data & Analytics - BNZ Confluence](#)

At a high level, each Zone will be represented in future by:

Technology	Per Zone specification	Comments
Snowflake	Databases per the data layer and database detailed design.	-
GitHub	One repository (with folder structures that represent each Subject Area).	Per the DevOps sub-design
Dbt Core		
Astronomer Airflow		

RBAC	Specific RBAC groupings aligned to each database in the Zone	Per the new Access section the Audit & Security sub-design
Jenkins	A Managed Jenkins instance per Zone	-

Optimization changes to these configurations will be delivered as part of the BDH Platform Uplift Wave 1 delivery and will be further described in detailed design below.

Detailed Design

Account Structure

In non-prod snowflake account, each database is aligned to a specific zone, layer and environment. Zones are useful to separate and independently manage the groupings of data products. Every zone will be created with standard configuration which will be automatically deployed during the time of zone creation. For example, there will be separate group of databases for loan and finance. Each group caters to its specific subject area while maintaining clear boundaries from the other. This alignment ensures that data that is relevant to one subject area is isolated which supports better governance.

Structure for Raw level

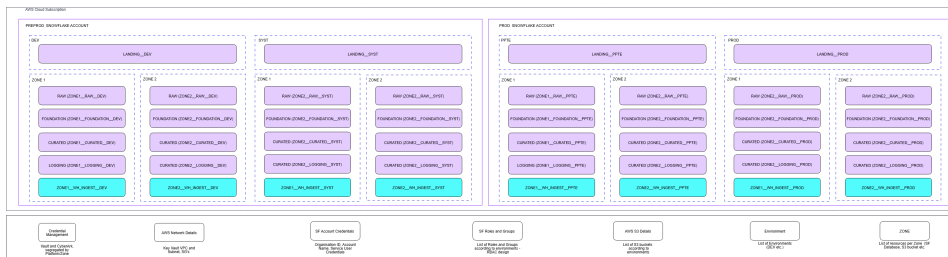
- RAW layer per Zone is represented by a single database
- By exception, a RAW layer source schema may be accessed by an FDP in another Zone
- Sources in the raw layer will be represented by a schema and access will be controlled at this level in raw as standard
- If data has a high confidentiality which necessitates specific access control, RBAC will be applied at a table level under the schema

Account Architecture

There are three Snowflake accounts currently in BDH.

- 1) NonProd - Existing pre-BDH Uplift non-prod Snowflake account where tenancy prior to BDH uplift exists. NonProd is also used for by DAP for platform work to develop / test infrastructure, updates, patches, new functionality etc. before it is applied to the (new) Pre-Prod SF Account (containing DEV & SYST) and the PROD SF account (containing PPTE & PROD)
- 2) PreProd - BDH Uplift's non-prod account, containing DEV and SYST environments
- 3) Prod - Existing Pre-BDH Uplift PROD account as well as BDH Uplift Prod account, containing PPTE and PROD. Objects will co-exist till decommissioned. Objects created by BDH Uplift is represented with specific naming conventions to separate the two, for ex: All AD groups as part of BDH will start with A_**BDH_SF_FG*** for AD groups created via SCIM or **BDH_*** for those created as part of RBAC natively in Snowflake for BDH.

The diagram below shows account architecture for PreProd and PROD snowflake accounts. Landing database will be common to all zones and other layers of a specific zone will have dedicated databases.



Other Databases

The databases so far referred to in the document specifies all the Databases that are part of the data movement. Apart from these, the following Databases will be part of BDHUplift and this is further detailed below.

Database	Full Database Name	Purpose	Reference Documents
PLATFORM__AUDIT__<ACCOUNT>	PLATFORM__AUDIT__PREPROD PLATFORM__AUDIT__PROD	Logging Database at Platform Level. Further details on Schemas under this database is documented in section: Logging Database Schemas	

GOVERNANCE__<ENV>	GOVERNANCE__DEV GOVERNANCE__SYST GOVERNANCE__PPTE GOVERNANCE__PROD	Database where Governance Objects are created, for Classification, Masking Policies and Tags.	
<ZONE>__LOGGING__<ENV>	Example: CUST__LOGGING__DEV CUST__LOGGING__SYST CUST__LOGGING__PPTE CUST__LOGGING__PROD	Further details on Schemas under this database is documented in section: Logging Database Schemas	
Databases Created From Shares	TBD	Any read-only shared database created from internal or external shares. Needs further discussion and alignment on naming conventions. Currently created as BNZ_DAP_PROD_AWS_DB_BDH_RAW_SYST02 BNZ_DAP_PROD_AWS_DB_BDH_RAW_SYST02 BNZ_DAP_PROD_AWS_DB_BDH_RAW_SYST02 Potential Naming Convention TBD	

Database Naming Convention

The table below outlines the physical database structure in matrix form by combining zones, environments and layers.

- Databases specific to an environment (which are part of a zone) can be identified using the name (ending with __DEV or __SYST)
- Database name will start with zone names, this will help to identify the zone to which a particular database belongs
- Database name will contain the layer name. This will help identify the layer a particular database belongs to



General

Double underscores (e.g. "__") are used to delimit zone, layer and environment (ZONE__LAYER__ENV), and generally where a name /function consists of more than one word.

		Account and Environments			
		PREPROD Account		PROD Account	
		DEV	SYST	PPTE	PROD
Data Layers	Land	LANDING__DEV	LANDING__SYST	LANDING__PPTE	LANDING__PROD
	Raw	<ZONE>__RAW__DEV	<ZONE>__RAW__SYST	<ZONE>__RAW__PPTE	<ZONE>__RAW__PROD
	Foundation	<ZONE>__FOUNDATION__DEV	<ZONE>__FOUNDATION__SYST	<ZONE>__FOUNDATION__PPTE	<ZONE>__FOUNDATION__PROD
	Curated	<ZONE>__CURATED__DEV	<ZONE>__CURATED__SYST	<ZONE>__CURATED__PPTE	<ZONE>__CURATED__PROD
	Transform	<ZONE>__TRANSFORM__DEV			
Logging	Zone Logging	<ZONE>__LOGGING__DEV	<ZONE>__LOGGING__SYST	<ZONE>__LOGGING__PPTE	<ZONE>__LOGGING__PROD
	Platform Audit and Logging	PLATFORM__AUDIT__PREPROD		PLATFORM__AUDIT__PROD	
Governance	Governance Database	GOVERNANCE__DEV	GOVERNANCE__SYST	GOVERNANCE__PPTE	GOVERNANCE__PROD
Shared Database		TBD		TBD	

<ZONE>__TRANSFORM__DEV

The <ZONE>__TRANSFORM__DEV is a special database that exists only in DEV. This enables development and end-to-end unit testing without raising conflicts with other developer's object execution during unit testing.

Every object built locally in dbt core, before it is executed in Astronomer, is created within a named schema under this database. Every developer will have a schema named after the developer, typically the BNZ User ID.

This allows developers to run end-to-end upstream and downstream models to perform complete validation without it affecting objects in the other data layers such as RAW/FOUNDATION/CURATED. When objects are deployed via Astronomer, these are created accordingly in RAW /FOUNDATION/CURATED.

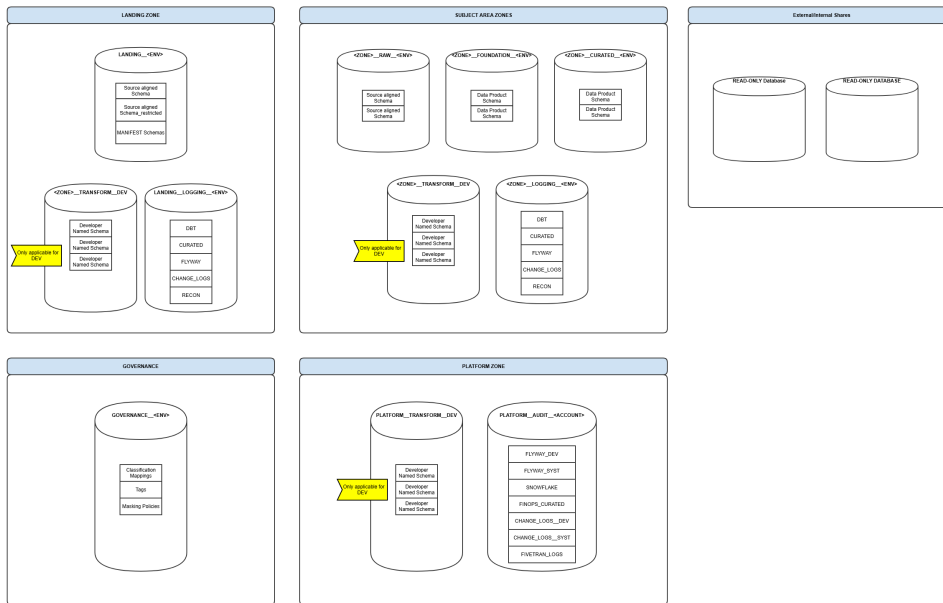
The override macros that define where objects are created from dbt (generate_alias_name, generate_database_name and generate_schema_name) have been refactored to consider this.

As a result, developers do not need to specify this explicitly in the dbt project configuration to differentiate the behavior in dbt and can simply follow the naming conventions required for a model to make this work for both TRANSFORM database as well as its final materialization in RAW /FOUNDATION/CURATED.

For further reading, please refer to the following document: [List of Macros & their detailed Specifications](#)

Comprehensive View of all the Databases in BDHUplift

The following diagram represents the full list of Databases as part of BDHUplift.



Logging Database Schemas

For the Platform logging and Audit database, as well as the <ZONE>__LOGGING__<ENV> databases that are available for each of the Zones, the following table describes a consistent pattern for the SCHEMAS created under these databases, and the purpose of each.

- Under PLATFORM__AUDIT__<ACCOUNT> databases, the following Schemas exists
 - FLYWAY__<ENV>
 - <ENV> is DEV and SYST for PREPROD <Account>
 - <ENV> is PPTE and PROD for PROD <Account>
 - SNOWFLAKE
 - FINOPS_CURATED
 - CHANGE_LOGS__<ENV>
 - <ENV> is DEV and SYST for PREPROD <Account>
 - <ENV> is PPTE and PROD for PROD <Account>
 - FIVETRAN_LOGS
- Under the <ZONE>__LOGGING__<ENV> database, the following Schemas exist
 - DBT
 - CURATED
 - RECON
 - FLYWAY
 - CHANGE_LOGS

Logging Database	Schema Name	Purpose
PLATFORM__AUDIT__<ACCOUNT>	FLYWAY[<ENV>]	Stores Platform Flyway migration history and status
PLATFORM__AUDIT__<ACCOUNT>	SNOWFLAKE	Snowflake Shared Database Views (from SNOWFLAKE database)

PLATFORM__AUDIT__<ACCOUNT>	FINOPS_CURATED	Curated Tables from Snowflake Views
PLATFORM__AUDIT__<ACCOUNT>	CHANGE_LOGS__<ENV>	Stores Platform Managed Jenkins logs
PLATFORM__AUDIT__<ACCOUNT>	FIVETRAN_LOGS	Logs from Fivetran (Note, this is created in PLATFORM__AUDIT__<ACCOUNT> as FIVETRAN logs are at Account Level)
<ZONE>__LOGGING__<ENV>	DBT	Contains Data Build Tool (dbt) execution logs and metadata
<ZONE>__LOGGING__<ENV>	CURATED	Maintains Curated view of dbt execution logs and metadata
<ZONE>__LOGGING__<ENV>	RECON	Hosts all 'ART - Automated Reconciliation and Testing Framework' artefacts.
<ZONE>__LOGGING__<ENV>	FLYWAY	Stores Zone-specific Flyway migration history and status
<ZONE>__LOGGING__<ENV>	CHANGE_LOGS	Stores Zone-specific Managed Jenkins logs

Warehouse Naming Convention

The table below provides the naming convention for virtual warehouses created for BDH Uplift.

- Warehouse name will start with zone/logical grouping, this will help to identify where the WH is being used. Example for Zones this will be the ZONE_NAME, for Platform it will be PLAT.
- Based on needs there can be multiple warehouses within the logical grouping, appropriate warehouse can be selected based on the purpose; WH_{purpose} provides unique name within zone
- Warehouse name will have a name part at the end, with one of the two options
 - Where a WH is created for an ENV within an ACCOUNT, this will end with the __<ENV> name part. An <ENV> could have values DEV/SYST/PPTE/PROD
 - Where a WH is created for an ACCOUNT, where there is no further ENV segregation, this will end with the __<ACCOUNT> name part. An <ACCOUNT> could have values PREPROD and PROD.

The Naming convention therefore being <FUNCTIONAL/SUBJECT_AREA>__WH__<PURPOSE>__<ENV/ACCOUNT>

Table below provides some samples.

Account	PREPROD		PROD	
Environment	DEV	SYST	PPTE	PROD
Zone Warehouses	<ZONE>__WH__<PURPOSE>__DEV	<ZONE>__WH__<PURPOSE>__SYST	<ZONE>__WH__<PURPOSE>__PPTE	<ZONE>__WH__<PURPOSE>__PROD
LANDING/INGEST Warehouses	LANDING__WH_KAFKA_INGEST__DEV	LANDING__WH_KAFKA_INGEST__SYST	LANDING__WH_KAFKA_INGEST__PPTE	LANDING__WH_KAFKA_INGEST__PROD
Alation	ALATION__WH_METADATAAEXTRACTION__DEV	ALATION__WH_METADATAAEXTRACTION__SYST	ALATION__WH_METADATAAEXTRACTION__PPTE	ALATION__WH_METADATAAEXTRACTION__PROD
Platform	PLAT__WH_GENERAL__DEV	PLAT__WH_GENERAL__SYST		
Platform	PLAT__WH_SPLUNK__PREPROD		PLAT__WH_SPLUNK__PROD	
Governance	GOVERNANCE__WH_GENERAL__DEV	GOVERNANCE__WH_GENERAL__SYST	GOVERNANCE__WH_GENERAL__PPTE	GOVERNANCE__WH_GENERAL__PROD

Resource Monitor Naming Convention

Resource Monitors in Snowflake help control costs and avoid unexpected credit usage caused by running warehouses. For every warehouse created, it is expected that an RM is attached to the warehouse. An Account level RM attached to ensure that across all the warehouses used by various Zones does not cause unintended credit usage. While this is true, individual WH are setup with RM specific for a Zone to allow for fine grained limits (which will be provisioned within the limit of what is set for the account).

Resource Monitors naming conventions are as follows, <FUNCTIONAL/SUBJECT_AREA>__RM__<ENV/ACCOUNT>

- Resource Monitors name will have a name part at the end, with one of the two options
 - Where an RM is created for an ENV within an ACCOUNT, this will end with the __<ENV> name part. An <ENV> could have values DEV/SYST/PPTE/PROD
 - Where an RM is created for an ACCOUNT, where there is no further ENV segregation, this will end with the __<ACCOUNT> name part. An <ACCOUNT> could have values PREPROD and PROD.

The following page captures the RM limits for each Zone that is setup: [Warehouse and Resource Manager Credit Limits - Digital, Data & Analytics - BNZ Confluence](#)

Example Purpose Names: 1. Source a Ingestion/Compute [CDC Topic] 2. API (Consumer) Source

Future State Zones

In BDH context, zones are logical grouping of databases intended to serve a specific subject area. A zone would contain three layers - RAW, foundation & curated.

Raw layer of a zone will contain different schemas which will be used to store raw data based on the data source in landing.

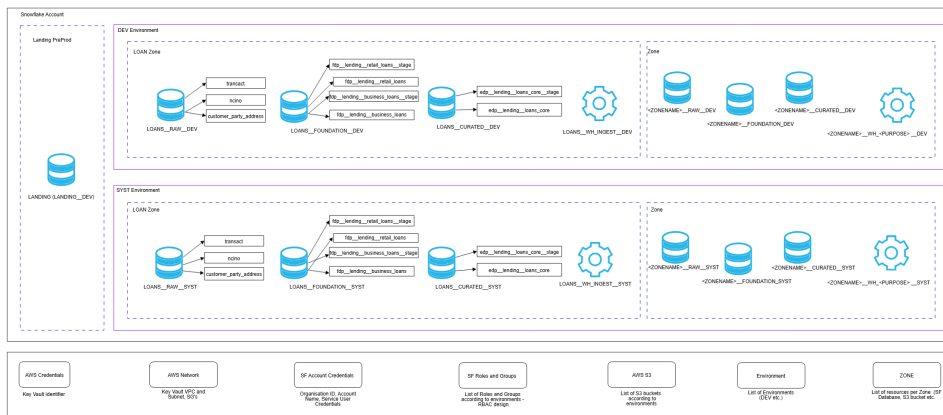
The database in foundation layer will host foundational data products. data products on the foundational layer will be deployed in separate schema Enterprise data products will be hosted in the curated layer database. To build the EDP, the data from FDP will be consumed

Zone level

- Subject areas will be represented by a zone, per data layer, in a single database, per SDLC environment
- Data-product-groups and data-products will be represented by schema naming conventions
- Data products are a grouping of technical assets under data product-groups
- A data product will be represented by a schema
- Data products may have an optional set of staging tables housed in a schema that will be housed by a schema with suffix "_stage"

Zone Architecture

The diagram below shows zone level segregation of databases for RAW, Foundation & Curated layers with example zone (and example systems) as "LOAN". Foundational and enterprise data products are in their own dedicated schemas within Foundation and Curated layers respectively.



Zone specification

Each Zone will be described by a YAML based specification which will be used by the deployment pipeline process to configure the desired characteristics and resources.

The YAML specification file will specify or allow deployment of the following:

Specification item	Description	Target system /component	Implementation specifics
Zone name	The name that the zone and its sub components will be identified by.	Various	User specified in YAML itself.
Product groups	The data product groups that will hold the data products within this zone.	Snowflake	Specified against the data products themselves in the YAML. Applies Data Product Group config (applies common Db settings and RBAC across any Db in the Group,
RBAC	The RBAC configuration for the zone.	Snowflake	Will call the Python scripts described in BDH sub-design - Audit & Security .
Databases	The databases that the zone will contain.	Snowflake	Create databases per the standards and structuring above. Will use the zone name entered into the specification. Tags specified in YAML.
Schemas	The schemas that the zone's databases will contain.	Snowflake	Create standard schemas in appropriate layers and environments (FDP - Dev RAW, Dev Foundation and Test, PPTE & PROD environments) (EDP or BDP - Dev Curated, Test Curated, PPTE & PROD) Tags specified in YAML.

Warehouses	The warehouses that the zone will contain.	Snowflake	Create any user specified warehouses per the naming standards above Specified in YAML: • Name • Compute size (for each SDLC env) • Query acceleration toggle • Tags to apply
Code Repositories	The code repositories that support the code used in the zone as well as dbtCore and Astronomer.	GitHub	Implemented based upon the Zone name.
CICD pipelines	The CICD pipelines that support the promotion and management of code used in the Zone.	Managed Jenkins	Implemented based upon the Zone name.



Future zone deployment inclusions

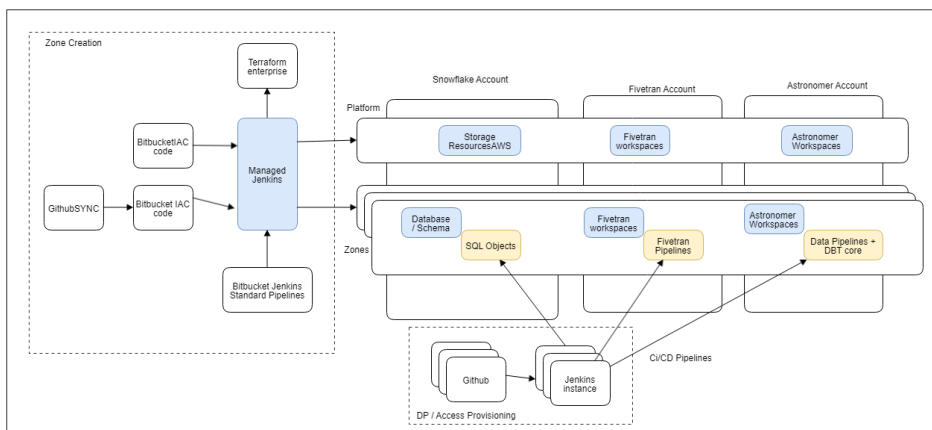
The following are slated for later iterations of this process:

- Update Metadata in Catalogue
- Create assets in LeanIX & ServiceNow
- Fivetran inclusions

Zone Deployment Approach

The diagram below shows deployment approach for databases.

- Each Zone will be having different Jenkins instance
- The aforementioned YAML specification file will be the reference point for deployment of the zone; with specific scripts & technologies used to execute what it requires.
- Each zone will manage code in separate Git repository and having pipelines in respective Jenkins instances.



Details of the deployment architecture can be referenced from [BDH HL sub-design - DevOps - Digital, Data & Analytics - BNZ Confluence](#)